

Analisi del Problema e Proposta di Soluzione

1. Contesto

Scenario

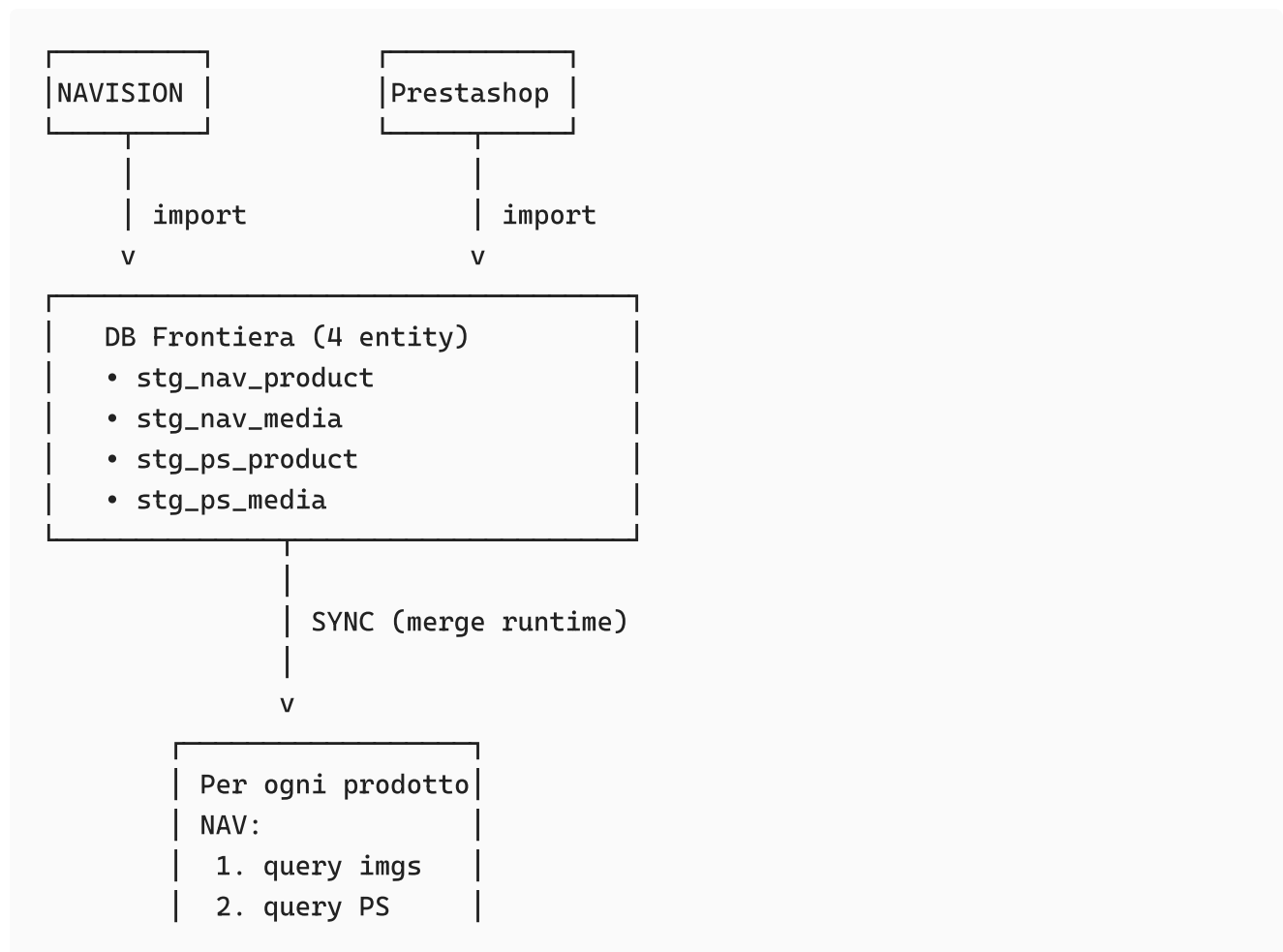
- **Sistema attuale:** Prestashop (vecchio e-commerce)
- **Sistema target:** WooCommerce (nuovo e-commerce)
- **Gestionale:** Microsoft Navision (NAV)

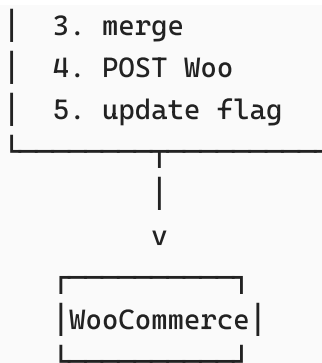
Obiettivo

Migrare prodotti da NAV a WooCommerce, arricchendoli con dati editoriali da Prestashop quando disponibili.

2. Architettura Attuale

Flusso operativo





Caratteristiche attuali

Import separati:

- Prodotti NAV → entity `stg_nav_product`
- Immagini NAV → entity `stg_nav_media`
- Prodotti Prestashop → entity `stg_ps_product`
- Immagini Prestashop → entity `stg_ps_media`
- Categorie → flusso indipendente

Sincronizzazione a runtime:

- Loop su tutti i prodotti NAV
- Per ogni prodotto:
 - Query sulle immagini (NAV/PS)
 - Query su Prestashop per arricchimento
 - Costruzione payload WooCommerce
 - Chiamata API WooCommerce
 - Aggiornamento flag nello staging

Dry-run:

- Flag booleano che salta chiamate HTTP, upload media, alcune letture file

3. Problemi Identificati

3.1 Merge durante la sincronizzazione

Sintomo: Per ogni prodotto si rieseguo query incrociate su 4 entity diverse.

Conseguenze:

- **Performance:** N+1 query (1 per prodotto)
- **Complessità cognitiva:** lo stato "vero" di un prodotto esiste solo durante il loop

- **Debug difficile:** "Perché questo prodotto ha questi dati?" richiede ricostruire la catena di query
- **Manutenibilità:** logica di merge sparsa nel codice di sync

3.2 Dry-run inadeguato

Sintomo: Dry-run usato come strumento di test, ma salta parti critiche del sistema.

Conseguenze:

- Non valida chiamate API reali (timeout, rate limit, autenticazione)
- Non testa upload media (permessi, formato, dimensioni)
- Non passa dai failure mode reali di WooCommerce
- **Falsa sicurezza:** "dry-run ok" non garantisce che la sync funzioni

3.3 Gestione immagini frammentata

Sintomo: Immagini arrivano da due fonti separate, gestite in entity diverse.

Conseguenze:

- Rischio duplicati (stesso URL/contenuto caricato più volte)
- Ordinamento instabile
- Cleanup difficile in caso di errore
- Nessun riuso tra prodotti

3.4 Testing rischioso

Sintomo: Manca ambiente sicuro per testare sync completa.

Conseguenze:

- Test su produzione = rischio cancellazioni/duplicati
- Nessun meccanismo di rollback
- Cleanup manuale laborioso
- Impossibile validare modifiche senza impatto

4. Soluzione Proposta: Architettura a 4 Fasi

Principio guida

Separare nettamente le fasi: non decidere com'è un prodotto mentre lo stai già scrivendo su WooCommerce.

Architettura target

5. Dettaglio delle 4 Fasi

FASE 1: IMPORT (invariato)

Obiettivo: Copiare dati grezzi da sorgenti esterne.

Cosa fa:

- Import NAV → `stg_nav_product` , `stg_nav_media`
- Import Prestashop → `stg_ps_product` , `stg_ps_media`
- Import categorie → `stg_categories`

Caratteristiche:

- Zero logica di business
 - Dati "as-is" dalle sorgenti
 - Può rimanere esattamente come oggi
-

FASE 2: CANONICAL (cuore della soluzione)

Obiettivo: Creare un'unica fonte di verità interna applicando regole di merge deterministiche.

Struttura dati

Tabella principale: `canonical_product`

Chiave: `sku` (business key)

Dati prodotto:

`name`, `short_description`, `long_description`
`price`, `stock_quantity`, `brand`
`attributes` (JSONB)

Governance:

`field_ownership` (JSONB)
→ `{"price": "nav", "long_description": "ps_frozen"}`

Controllo:

`payload_hash` (SHA256 per idempotenza)
`sync_status` (NEW | READY | SENT | FAILED)

last_merged_at

ID esterni:

nav_id, ps_id, woo_product_id

Tabella media: canonical_product_media

sku (FK a canonical_product)
media_key (hash contenuto o URL normalizzato)
source (nav | ps)
role (main | gallery)
sort_order
alt_text
woo_media_id (quando caricato)

UNIQUE(sku, media_key) → dedup automatico

Tabella mapping: id_map

sku (PK)
nav_id
ps_id
woo_product_id

Regole di merge

Campo	Fonte Master	Razionale
Prezzo	NAV	Gestionale è fonte di verità per dati transazionali
Stock	NAV	idem
Brand	NAV	Dato strutturato gestionale
Nome	Prestashop → NAV	Contenuto editoriale preferito, fallback NAV
Descrizioni	Prestashop → vuoto	Solo se disponibile arricchimento
Immagini	Prestashop first, NAV second	Qualità editoriale, dedup per hash

Processo di merge

Per ogni SKU in stg_nav_product:

1. Cerca corrispondenza in stg_ps_product (via SKU/EAN)
2. Applica regole di priorità campo per campo
3. Raccogli immagini PS + NAV, dedup per hash URL
4. Calcola payload_hash

5. Upsert in canonical_product
6. Insert in canonical_product_media (deduplicate)

Benefici:

- **Fonte unica:** "Com'è questo prodotto?" = SELECT su 1 tabella
- **Merge 1 volta:** non ripetuto ad ogni sync
- **Debug semplice:** dati + metadati in un posto solo
- **Testabilità:** logica di merge testabile separatamente

FASE 3: PLAN (decisioni senza effetti)

Obiettivo: Decidere cosa cambiare su WooCommerce senza fare modifiche.

Struttura piano

Tabella: sync_plan

```
run_id (identificativo esecuzione)
sku
action (CREATE | UPDATE | NOOP)
payload_hash (per confronto idempotente)
media_actions (JSONB)
  → [{"media_key": "abc", "action": "upload|reuse"}]
dependencies (JSONB)
  → ["media:xyz", "category:123"]
status (PENDING | SUCCESS | FAILED)
retry_count
last_error
executed_at
```

Processo

Per ogni sku in canonical_product con sync_status=READY:

1. Recupera woo_product_id da id_map (se esiste)
2. Se esiste:
 - Confronta payload_hash
 - Se diverso → action=UPDATE
 - Se uguale → action=NOOP

Altrimenti:

- action=CREATE
3. Per ogni media in canonical_product_media:
 - Controlla se woo_media_id esiste
 - Se no → media_action=upload

```
- Se sì → media_action=reuse  
4. Insert in sync_plan
```

Dry-run corretto

Dry-run = genera PLAN completo e si ferma prima di EXECUTE.

Non salta logica:

- Costruisce payload WooCommerce reale
- Valida mapping categorie
- Risolve dipendenze media
- Calcola hash

Output:

- Tabella sync_plan popolata
- Report: "45 CREATE, 203 UPDATE, 12 NOOP, 89 media upload"
- Validazioni: "5 SKU duplicati, 3 categorie non mappate"

Vantaggi:

- Visibilità completa su impatto
- Validazione pre-esecuzione
- Plan persistito (debug post-mortem)
- Non è un "finto test"

FASE 4: EXECUTE (effetti reali)

Obiettivo: Applicare il piano su WooCommerce.

Processo

Per ogni item in sync_plan con status=PENDING:

1. Upload media (con riuso)

Per ogni media_action:

- Se action=upload:

- Download contenuto
- POST /wp/v2/media
- Update canonical_product_media.woo_media_id

- Se action=reuse:

- Recupera woo_media_id esistente

2. Create/Update prodotto

- Se action=CREATE:

- POST /wc/v3/products
- Update id_map.woo_product_id
- Se action=UPDATE:
 - PUT /wc/v3/products/{id}

3. Aggiorna stato

- canonical_product.sync_status = SENT
- sync_plan.status = SUCCESS
- sync_plan.executed_at = now()

Idempotenza

Media: media_key → woo_media_id mapping impedisce duplicati.

Prodotti: payload_hash evita update inutili.

Ripartenza: se execute fallisce, riparti da sync_plan con status=PENDING.

6. Confronto Architetture

Aspetto	Architettura Attuale	Architettura Proposta
Complessità query	N+1 (query per prodotto)	Batch join in merge → read 1 tabella
Dove è la verità	Ricostruita a runtime	Tabella canonical_product
Debug	Rieseguire catena query	SELECT su canonical + source_flags
Idempotenza	Gestita manualmente	payload_hash + mapping automatici
Immagini duplicate	Possibili	Impossibili (UNIQUE su media_key)
Dry-run	Salta chiamate = non testa	Genera plan = valida tutto
Testing	Rischioso (prod o logica finta)	Sicuro (staging con marker)
Manutenibilità	Logica sparsa	Fasi separate, chiare

7. Strategia di Testing

Livello 1: Test Unitari

Cosa: Funzioni pure di merge.

Senza: Database, API, filesystem.

Esempio:

```
Input:  nav_product(price=10), ps_product(price=12)
Output: canonical(price=10)  // NAV master
```

Velocità: Centinaia al secondo.

Copertura: Regole priorità, dedup, normalizzazione.

Livello 2: Test Integrazione

Cosa: Pipeline completa staging → canonical.

Con: Database reale (container).

Senza: WooCommerce.

Esempio:

```
Setup:  Insert 3 nav_product, 2 ps_product
Run:    Merge job
Assert: 3 righe in canonical_product con arricchimenti corretti
```

Velocità: Secondi.

Copertura: Join SQL, mapping, gestione duplicati.

Livello 3: Test E2E

Cosa: Sync completa su WooCommerce staging.

Con: Ambiente Woo reale, chiamate API vere.

Strategia marker:

- SKU test: TST-{run_id}-{sku} (es. TST-20250113-PROD001)
- Oppure meta field: _test_run_id: uuid

Cleanup automatico:

```
Alla fine del test:
1. GET /wc/v3/products?sku=TST-{run_id}-*
```

2. DELETE force=true per ogni prodotto
3. DELETE media associati

Esempio:

Test: Crea prodotto con 2 immagini
Verifica via GET che esista
Verifica gallery ordinata
Ri-sync (idempotenza)
Cleanup

Velocità: Minuti (pochi test critici).

Copertura: API reale, media upload, errori Woo, idempotenza.

Il ruolo del dry-run

Dry-run NON sostituisce i test.

Strumento	Scopo	Quando usare
Dry-run	Preview, validazione dati	Prima di ogni sync in prod
Test unitari	Verifica logica merge	Sviluppo, CI/CD
Test integrazione	Verifica pipeline staging→canonical	Sviluppo, CI/CD
Test E2E	Verifica comportamento reale Woo	Prima rilascio, regressioni

8. Piano di Migrazione Incrementale

Obiettivo

Passare alla nuova architettura **senza downtime** e **senza big-bang**.

Step 1: Setup Canonical (settimana 1)

Attività:

- Crea tabelle `canonical_product`, `canonical_product_media`, `id_map`
- Implementa job merge: staging → canonical
- Esegui merge iniziale (popolamento)

Stato sistema:

- ☒ Canonical popolato e aggiornato
- ☒ Sync attuale continua a funzionare (legge da staging)
- ☒ Nessuno usa ancora canonical

Rischio: Basso (nessuna modifica a sync esistente)

Step 2: Refactor Sync (settimana 2)

Attività:

- Modifica sync per leggere da `canonical_product` invece di staging
- Elimina query incrociate nel loop
- Mantieni stessa logica chiamate Woo

Stato sistema:

- ☒ Sync legge da canonical
- ☒ Performance migliorate (no N+1)
- ☒ Debug più semplice
- ☒ Plan non ancora introdotto

Rischio: Medio (modifica sync, ma logica invariata)

Rollback: Ripristina sync a leggere da staging

Step 3: Introduce Plan (settimana 3)

Attività:

- Crea tabella `sync_plan`
- Implementa fase PLAN (canonical → plan)
- Modifica EXECUTE per leggere da plan
- Dry-run ora genera plan e si ferma

Stato sistema:

- ☒ Plan persistito
- ☒ Dry-run validante
- ☒ Tracciamento esecuzioni
- ☒ Test E2E non ancora setup

Rischio: Medio (modifica flusso, ma funzionalità invariata)

Rollback: Elimina fase plan, sync legge direttamente da canonical

Step 4: Setup Testing (ongoing)

Attività:

- Provisioning WooCommerce staging
- Implementa marker test (`TST-{uuid}-`)
- Script cleanup automatico
- Test E2E iniziali (3-5 casi critici)

Stato sistema:

- ☒ Testing affidabile
- ☒ Confidence su modifiche future
- ☒ Documentazione aggiornata

Rischio: Basso (non impatta produzione)

Step 5: Ottimizzazioni (settimana 4+)

Attività:

- Batch processing per grandi volumi
- Retry automatici con backoff
- Monitoring e alerting
- Test unitari estesi

Rischio: Basso (incrementale)

9. Decisione: Quando Usare Canonical

Usa architettura canonical se:

- ☒ **Multiple sorgenti dati** da mergiare (nel tuo caso: NAV + Prestashop)
- ☒ **Regole di merge complesse** (priorità, fallback, conditional logic)
- ☒ **Idempotenza critica** (rilanci frequenti, possibili interruzioni)

- ✅ **Debug importante** (supporto, troubleshooting produzione)
 - ✅ **Lungo termine** (manutenibilità > velocità iniziale sviluppo)
 - ✅ **Team distribuito** (chiarezza architetturale essenziale)
-

Evita canonical se:

- ❌ **Una sola sorgente dati** (es. solo NAV)
 - ❌ **Sync "dump diretto"** senza logica (copia 1:1)
 - ❌ **Job one-off** (migrazione unica, poi mai più)
 - ❌ **Prototipo rapido** (validazione business, non produzione)
-

Nel tuo caso specifico

Hai:

- 2 sorgenti (NAV + Prestashop)
- Merge con priorità campo per campo
- Immagini da deduplicare
- Sistema long-running (non one-off)
- Necessità di manutenibilità

Conclusione: ✅ **Canonical è la scelta corretta.**

10. Sintesi Esecutiva

Il problema

Oggi il sistema **mescola import, merge e sync**, usando dry-run come falsa rete di sicurezza. Risultato: logica sparsa, debug difficile, testing rischioso.

La soluzione

4 fasi separate: IMPORT → CANONICAL → PLAN → EXECUTE.

Pilastri:

1. **CANONICAL:** fonte unica interna (merge una volta)
2. **PLAN:** dry-run validante (genera piano, non salta logica)

3. **EXECUTE**: effetti reali testati su staging

Il beneficio principale

Non è velocità o performance — è **sostenibilità mentale**.

Rispondere "com'è questo prodotto?" guardando 1 tabella invece di ricostruire 4 query cambia: debug, onboarding, manutenzione, evoluzione futura.

Migrazione

Incrementale: 4 step settimanali, sync attuale continua a funzionare. Zero downtime.

Messaggio chiave

Il problema non è il dry-run — è che viene usato per compensare un'architettura che mescola troppe responsabilità. Separando le fasi con uno stato canonical chiaro, il sistema diventa semplice, testabile e manutenibile.

Appendice: Riferimenti Rapidi

Checklist decisionale

- ☐ Hai multiple sorgenti dati?
- ☐ Serve merge con regole complesse?
- ☐ Idempotenza è critica?
- ☐ Debug/troubleshooting frequente?
- ☐ Sistema long-running (non one-off)?

Se ≥3 "sì" → Canonical è appropriato.

Segnali di warning architettura attuale

- "Per capire perché questo prodotto è così, devo rieseguire query"
- "Dry-run ok ma sync in prod fallisce"
- "Non posso testare senza rischiare danni"
- "Le immagini si duplicano spesso"
- "Onboarding nuovo dev richiede settimane"

Se ≥2 warning → Refactoring giustificato.

Metriche di successo post-migrazione

- ⚡ **Debug time**: da ore a minuti
- ⚡ **Test confidence**: da 60% a 95%+

- ⚡ **Onboarding:** da 3 settimane a 1 settimana
 - ⚡ **Incident rate:** -70%
 - ⚡ **Query count:** -90% (no N+1)
-